

FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA — FIAP

Pós-Graduação em Inteligência Artificial para Desenvolvedores

ASSISTENTE MÉDICO VIRTUAL COM FINE-TUNING, RAG E FLUXOS CLÍNICOS AUTOMATIZADOS

Fine-tuning com LoRA/PEFT, RAG e orquestração LangChain/LangGraph aplicados à decisão clínica assistida

Tech Challenge — Fase 3

Wilson Lima da Silva | wilson.slima@gmail.com

Gustavo Lopes da Silva | gustavo_lsilva@hotmail.com

Felipe Soeiro Lopes | felipesoeiro.contato@outlook.com.br

Vinicius Tavares Sousa da Silva | viniciustavares2014@gmail.com

São Paulo

2026

RESUMO

O presente trabalho descreve o desenvolvimento de um assistente médico virtual voltado ao apoio à decisão clínica em ambiente hospitalar, treinado com dados internos sintéticos (protocolos clínicos, FAQs de médicos e modelos de laudos) e complementado por uma amostra real do dataset público PubMedQA. O sistema combina três frentes de Inteligência Artificial aplicada: (i) fine-tuning supervisionado com LoRA (Low-Rank Adaptation) sobre um modelo base leve (distilgpt2), com suporte documentado para arquiteturas LLaMA reais (TinyLlama-1.1B) em ambientes com GPU; (ii) uma camada de Retrieval-Augmented Generation (RAG) sobre os protocolos internos, associada a uma base estruturada de prontuários em SQLite para contextualização por paciente; e (iii) um fluxo de decisão clínica automatizado, orquestrado com LangGraph, que verifica exames pendentes, consulta protocolos, sugere condutas e emite alertas para a equipe médica. O assistente nunca prescreve diretamente — toda sugestão é sinalizada como rascunho sujeito a validação humana — e conta com guardrails de segurança, bloqueio de tópicos fora de escopo e logging de auditoria detalhado para cada interação. Três provedores de LLM intercambiáveis foram implementados (determinístico, fine-tuned local e OpenAI), com seleção automática do modo mais seguro e reprodutível quando nenhuma chave de API ou GPU está disponível. O sistema é validado por 26 testes automatizados e disponibiliza CLI, API HTTP e interface web de chat para uso interativo.

Palavras-chave: fine-tuning; LoRA; PEFT; LangChain; LangGraph; RAG; assistente médico virtual; segurança em IA clínica; PubMedQA.

SUMÁRIO

1 INTRODUÇÃO

2 FUNDAMENTAÇÃO TEÓRICA

- 2.1 Fine-tuning e LoRA/PEFT
- 2.2 Retrieval-Augmented Generation (RAG)
- 2.3 Orquestração com LangChain e LangGraph
- 2.4 Segurança e Guardrails em IA Clínica

3 METODOLOGIA

4 IMPLEMENTAÇÃO

- 4.1 Estrutura do Projeto
- 4.2 Dados e Curadoria
- 4.3 Pipeline de Fine-tuning
- 4.4 Arquitetura do Assistente (RAG + LangChain)
- 4.5 Fluxo Clínico Automatizado (LangGraph)
- 4.6 Segurança, Guardrails e Auditoria
- 4.7 Interface Web e API

5 AVALIAÇÃO E RESULTADOS

6 RESULTADOS CONSOLIDADOS

REFERÊNCIAS

 **Repositório GitHub**

https://github.com/tech-challenge-fiap-GRUPO-02-2026/tech_challenge_3

 **Página de Apresentação do Projeto (HTML)**

https://tech-challenge-fiap-grupo-02-2026.github.io/tech_challenge_3/

1 INTRODUÇÃO

A atuação clínica em ambiente hospitalar exige acesso rápido e confiável a protocolos internos, histórico do paciente e evidência médica atualizada. A sobrecarga de informação e a pressão do tempo de atendimento tornam esse acesso um gargalo operacional relevante, com impacto direto na qualidade e segurança do cuidado. Assistentes virtuais baseados em Large Language Models (LLMs) surgem como uma abordagem promissora para reduzir esse gargalo, desde que operados sob controles rígidos de segurança, explicabilidade e auditoria — requisitos inegociáveis em contexto médico.

Este trabalho apresenta o desenvolvimento do Projeto da Fase 3 do Tech Challenge da Pós-Graduação em Inteligência Artificial para Desenvolvedores da FIAP, dando continuidade ao projeto de otimização de rotas médicas da Fase 2. Enquanto a Fase 2 tratou da otimização combinatória da distribuição logística de medicamentos, esta fase desloca o foco para a camada de IA generativa aplicada à decisão clínica: um assistente médico virtual treinado com dados internos do hospital (protocolos, FAQs e modelos de laudos), capaz de responder dúvidas clínicas citando a fonte, consultar prontuários estruturados e executar um fluxo de decisão automatizado que nunca substitui o julgamento médico.

As principais contribuições deste trabalho incluem: (i) pipeline de fine-tuning com LoRA/PEFT reproduzível em qualquer CPU, com suporte documentado a arquiteturas LLaMA reais em GPU; (ii) camada RAG testável sobre protocolos clínicos internos, com fallback leve (TF-IDF) e modo avançado (FAISS/LangChain); (iii) integração de um dataset público real (PubMedQA) traduzido automaticamente para português, ampliando o escopo de resposta do assistente além dos protocolos internos; (iv) fluxo de decisão clínica automatizado com LangGraph, cobrindo da verificação de exames pendentes à emissão de alertas para a equipe; e (v) arquitetura de segurança com guardrails, bloqueio de escopo e auditoria completa de cada interação.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Fine-tuning e LoRA/PEFT

Fine-tuning é o processo de continuar o treinamento de um modelo de linguagem pré-treinado sobre um conjunto de dados específico de domínio, ajustando seus pesos para melhorar o desempenho em tarefas ou vocabulário especializados. LoRA — Low-Rank Adaptation (Hu et al., 2021) — é uma técnica de Parameter-Efficient Fine-Tuning (PEFT) que congela os pesos originais do modelo e injeta pequenas matrizes de baixo posto treináveis em camadas específicas (tipicamente as projeções de atenção), reduzindo drasticamente o número de parâmetros treináveis e o custo computacional do ajuste, sem a necessidade de reescrever o modelo inteiro.

No contexto deste projeto, o LoRA é aplicado sobre um modelo base leve (distilgpt2, 82M parâmetros) para viabilizar um treinamento reprodutível em qualquer CPU em segundos, mantendo ao mesmo tempo compatibilidade documentada com arquiteturas LLaMA reais (TinyLlama-1.1B-Chat), cujos módulos de atenção diferem estruturalmente do GPT-2 e exigem detecção automática por família de arquitetura.

2.2 Retrieval-Augmented Generation (RAG)

RAG é uma técnica que combina um mecanismo de recuperação de informação (retrieval) com um modelo gerador de linguagem, de modo que a geração de texto seja fundamentada (grounded) em documentos relevantes recuperados de uma base de conhecimento, em vez de depender exclusivamente do conhecimento paramétrico do modelo. Essa abordagem reduz alucinações e permite citar explicitamente a fonte utilizada — um requisito central em aplicações médicas, onde a rastreabilidade da informação é tão importante quanto a resposta em si.

O RAG deste projeto opera, por padrão, com um vetor TF-IDF local (sem dependências externas), podendo ser substituído por um índice FAISS via LangChain quando as dependências avançadas estão instaladas — mantendo a mesma interface pública em ambos os modos.

2.3 Orquestração com LangChain e LangGraph

LangChain é um framework para composição de pipelines de aplicações baseadas em LLMs, encadeando etapas como recuperação de contexto, formatação de prompt, chamada ao modelo e pós-processamento. LangGraph estende esse paradigma para fluxos com estado e ramificações condicionais, modelados como grafos direcionados de nós — adequado para representar processos de decisão clínica com múltiplas etapas sequenciais e checagens de segurança.

Neste projeto, o LangChain organiza o pipeline do assistente (RAG + contexto do paciente + geração de resposta), enquanto o LangGraph modela o fluxo clínico automatizado como uma sequência de nós com efeitos colaterais auditáveis (consulta a exames, sugestão de conduta, revisão de segurança e emissão de alertas).

2.4 Segurança e Guardrails em IA Clínica

Guardrails são mecanismos de controle que restringem o comportamento de um sistema de IA generativa, bloqueando entradas fora de escopo e sanitizando saídas potencialmente inseguras antes de chegarem ao usuário final. Em aplicações clínicas, a literatura recomenda que sistemas de apoio à decisão nunca substituam o julgamento do profissional de saúde, devendo toda sugestão automatizada ser explicitamente sinalizada como rascunho sujeito a validação humana — princípio adotado como requisito central deste projeto.

3 METODOLOGIA

O projeto foi conduzido de forma incremental, evoluindo por camadas independentes e testáveis isoladamente: (1) dados e curadoria, (2) fine-tuning, (3) RAG e prontuário estruturado, (4) orquestração via LangChain/LangGraph, e (5) segurança e auditoria. Cada camada foi desenvolvida com fallback determinístico e leve, garantindo que o sistema completo — incluindo os 26 testes automatizados — seja executável em qualquer máquina, sem GPU e sem chave de API.

- **Dados e curadoria** — criação de três fontes internas sintéticas (protocolos, FAQs, modelos de laudo) e download/tradução de uma amostra real do PubMedQA, seguidos de anonimização de prontuários com PII e montagem do dataset de instruction-tuning.
- **Fine-tuning** — treinamento supervisionado com LoRA sobre o dataset curado, com avaliação em conjunto de validação hold-out.
- **RAG e prontuário** — indexação dos protocolos para recuperação por similaridade e criação de uma base SQLite estruturada de pacientes e exames, já anonimizada.
- **Orquestração** — composição do assistente médico (LangChain) integrando RAG, prontuário e o provider de LLM selecionado, e modelagem do fluxo clínico automatizado (LangGraph).
- **Segurança e auditoria** — implementação de guardrails de bloqueio e sanitização, explicabilidade por citação de fontes, e logging estruturado de cada evento.

A validação foi conduzida com testes automatizados usando pytest ao longo de todo o desenvolvimento, cobrindo anonimização, curadoria, RAG, guardrails, prontuário, assistente e fluxo clínico. A última validação registrada apresentou **26 testes passando**.

4 IMPLEMENTAÇÃO

4.1 Estrutura do Projeto

src/	
data_processing/	- anonimização (PII) e curadoria do dataset
finetuning/	- config, treino LoRA/PEFT e avaliação
llm/	- provider abstrato (determinístico / fine-tuned / OpenAI)
rag/	- vector store (TF-IDF local ou FAISS/LangChain) + retriever
database/	- repositório SQLite de prontuários (já anonimizados)
langchain_app/	- pipeline do assistente médico (LangChain)
langgraph_flow/	- fluxo clínico automatizado (LangGraph)
safety/	- guardrails de segurança + logging de auditoria
explainability/	- formatação de citações e fontes
webapp/	- backend HTTP (stdlib) + front-end de chat
main.py	- CLI unificada
tests/	- 26 testes automatizados (pytest)
data/	
raw/protocolos/	- 3 protocolos clínicos internos (markdown)
raw/faqs/	- FAQs de médicos (jsonl)
raw/laudos/	- modelos de laudo/receita/evolução
raw/prontuarios_brutos.csv	- prontuários COM PII (entrada da anonimização)
processed/	- dataset de fine-tuning + prontuários anonimizados
prontuarios/	- base estruturada em runtime (pacientes.csv, exames.csv)
artifacts/	- saída de treino, avaliação e SQLite (gerados)
logs/	- logs de auditoria (gerados)
reports/	- relatório técnico consolidado (final_report.md)
docs/	- arquitetura, roteiro e guia de vídeo

4.2 Dados e Curadoria

Quatro fontes internas sintéticas foram criadas em `data/raw/`, complementadas por uma amostra real do dataset público PubMedQA (sugerido no enunciado do desafio):

Tabela 1 — Fontes de Dados e Exemplos Gerados

Fonte	Conteúdo	Exemplos
protocolos/*.md	3 protocolos clínicos completos (dor torácica, sepse, hiperglicemia) — sintéticos	17
faqs/faqs_medicos.jsonl	Perguntas frequentes de médicos, com resposta e fonte — sintéticas	10
laudos/modelos_laudos.md	Modelos estruturados de laudo, receita, evolução e solicitação de exame — sintéticos	4
pubmedqa/pubmedqa_sample.jsonl	Amostra real do PubMedQA (Jin et al., 2019), baixada via Hugging Face Hub (qiaojin/PubMedQA, config pqa_labeled) e traduzida para PT-BR (Helsinki-NLP/opus-mt-tc-big-en-pt, MarianMT local)	40

Fonte: elaborado pelos autores.

Os quatro primeiros são combinados por `src/data_processing/dataset_builder.py` em **71 exemplos** de instruction-tuning (formato Alpaca-like), salvos em `data/processed/finetune_dataset.jsonl`. O texto original em inglês do PubMedQA é preservado em `pubmedqa_sample_en.jsonl` para auditoria da tradução. A amostra traduzida também entra no pool de perguntas respondidas pelo provider determinístico, ampliando o escopo do assistente para além dos protocolos internos, sempre citando a fonte PubMedQA (`pubmedqa.github.io`) separadamente das fontes internas do hospital.

4.2.1 Anonimização de Prontuários

`src/data_processing/anonymize.py` processa `prontuarios_brutos.csv` (com PII sintética — nome, CPF, telefone, e-mail, endereço, convênio) em duas camadas: (1) remoção estrutural das seis colunas identificadoras diretas; (2) varredura por expressão regular em campos de texto livre remanescentes, mascarando CPF, telefone e e-mail residuais. Apenas o `paciente_id` (pseudônimo) é mantido como chave de junção, produzindo `prontuarios_anonimizados.csv`.

4.3 Pipeline de Fine-tuning

`src/finetuning/train.py` realiza fine-tuning supervisionado com LoRA via `peft`. O modelo base padrão é **distilgpt2** (82M parâmetros), treinando em aproximadamente 90 segundos em qualquer CPU — escolha deliberada para reprodutibilidade em qualquer máquina avaliadora, sem espera longa nem dependência de GPU.

O pipeline também suporta a troca do modelo base para uma arquitetura LLaMA real via `--base-model TinyLlama/TinyLlama-1.1B-Chat-v1.0` (1,1B parâmetros, aberto no Hugging Face, sem exigência de licença/token como o Llama 3 oficial da Meta). Os módulos de destino do LoRA são auto-detectados por família de arquitetura em `default_lora_target_modules()`: `c_attn` para GPT-2 e `q_proj/k_proj/v_proj/o_proj` para LLaMA/Mistral. Uma tentativa de execução do TinyLlama nesta máquina, apenas em CPU (sem GPU), passou de 6 horas de processamento sem completar sequer um dos dez passos configurados e terminou com o desligamento espontâneo do computador por sobrecarga — evidenciando a inviabilidade dessa arquitetura como padrão sem aceleração de hardware. Diante disso, a equipe manteve o `distilgpt2` como modelo padrão do projeto. A capacidade de treinar arquiteturas LLaMA reais permanece documentada e testável por qualquer avaliador com GPU disponível (NVIDIA/CUDA ou AMD/ROCm), com scripts de instalação dedicados em `scripts/`.

Tabela 2 — Hiperparâmetros do Fine-tuning LoRA

Parâmetro	Valor
Rank (r)	8
Alpha	16
Dropout	0,05
Épocas	3
Batch size	2
Learning rate	2e-4

Fonte: `src/finetuning/config.py`.

O adaptador treinado é salvo em `artifacts/finetuned_model/`, junto com `training_manifest.json`, contendo metadados reprodutíveis (modelo base, número de exemplos e hiperparâmetros utilizados).

4.3.1 Três Provedores de LLM Intercambiáveis

Tabela 3 — Providers de LLM (src/llm/provider.py)

Provider	Quando é usado	Detalhes
DeterministicMedicalProvider	Padrão do modo auto, sempre disponível	Casa a pergunta com a base de FAQs curadas por similaridade lexical (Jaccard) e responde com a fonte exata. Garante execução em qualquer máquina, sem GPU nem chave de API.
OpenAIProvider	Se OPENAI_API_KEY estiver definida	Usa gpt-4o-mini de verdade, com o mesmo contexto (protocolos + prontuário) injetado no prompt.
LocalFineTunedProvider	Somente quando pedido explicitamente (--provider local_finetuned)	Modelo base distilgpt2 ajustado via LoRA/PEFT com os 71 exemplos curados.

Fonte: elaborado pelos autores.

O modo auto nunca escolhe automaticamente o modelo fine-tuned: poucos passos de treino e um dataset pequeno (71 exemplos) não são suficientes para aderência clínica confiável. O modelo fine-tuned só entra em cena quando explicitamente selecionado, para fins de inspeção e demonstração do pipeline de fine-tuning.

4.3.2 Avaliação do Modelo

src/finetuning/evaluate.py separa um conjunto de validação hold-out (últimas 3 FAQs, não usadas no treino) e mede overlap lexical entre resposta gerada e resposta de referência, além de perplexidade quando o modelo fine-tuned está disponível via transformers.

Tabela 4 — Resultado da Avaliação (baseline determinístico)

Métrica	Resultado
Overlap lexical médio	1,0 (3/3 perguntas de hold-out corretamente recuperadas)
Perguntas avaliadas	3

Fonte: artifacts/eval_report.json.

Esse resultado representa o piso de qualidade do sistema: mesmo sem o modelo fine-tuned carregado, o assistente responde corretamente com base na curadoria realizada. O modelo fine-tuned é avaliado com a mesma interface (answer_question), permitindo comparação direta assim que treinado.

4.4 Arquitetura do Assistente (RAG + LangChain)

src/langchain_app/medical_assistant.py implementa a classe MedicalAssistant, que integra em um único pipeline: (1) RAG sobre os protocolos internos, com TF-IDF local por padrão ou FAISS via LangChain quando as dependências opcionais estão instaladas; (2) consulta à base estruturada de prontuários (SQLite, já anonimizada) via PatientRepository; (3) geração de resposta pelo provider de LLM selecionado; (4) guardrails de segurança; (5) explicabilidade, citando as fontes usadas e o contexto do paciente considerado; e (6) logging de auditoria de cada evento.

```

flowchart LR
    A[Protocolos, FAQs, modelos de laudo] --> B[Anonimizacao + Curadoria]
    B --> C[Fine-tuning LoRA (PEFT)]
    A --> D[RAG (TF-IDF / FAISS)]
    E[(SQLite: Prontuarios anonimizados)] --> F
    D --> F[MedicalAssistant (LangChain)]
    C -.modelo fine-tuned.-> F
    F <--> G[clinical_flow (LangGraph)]
    F --> H[Guardrails + Explainability]
    G --> I[Audit Logger (logs/audit_log.jsonl)]
    H --> I

```

Figura 1 — Arquitetura geral do sistema. Fonte: docs/architecture.md.

Toda resposta do assistente segue um formato padronizado: (1) a resposta em si, baseada na fonte que casou com a pergunta; (2) um aviso obrigatório de que se trata de sugestão de apoio à decisão sujeita a validação médica; (3) a fonte principal utilizada — um FAQ interno com seu protocolo de origem, ou o PubMedQA; (4) contexto adicional de protocolos internos relacionados, recuperado pelo RAG, apresentado apenas quando há similaridade relevante; e (5) o contexto do paciente considerado, quando um `--patient-id` é informado.

4.5 Fluxo Clínico Automatizado (LangGraph)

O fluxo de decisão automatizado (`src/langgraph_flow/clinical_flow.py`) coordena, via LangGraph, seis etapas sequenciais para um paciente informado:

Tabela 5 — Nós do Fluxo Clínico (LangGraph)

Nó	Responsabilidade	Saída
intake	Recebe o <code>paciente_id</code> e busca o registro no prontuário	<code>patient_found</code> , <code>diagnosis</code>
verificar_exames_pendentes	Consulta a tabela exames no SQLite	<code>pending_exams</code>
consultar_protocolo	Recupera trechos relevantes dos protocolos (RAG) para o diagnóstico	<code>protocol_guidance</code>
sugerir_conduta	Gera sugestão de próximos passos via LLM (nunca prescrição final)	<code>suggestion</code> (rascunho)
revisao_seguranca	Aplica guardrails — remove linguagem prescritiva direta, adiciona aviso obrigatório	<code>guardrail_flags</code> , <code>suggestion</code> sanitizada
emitir_alertas	Gera alertas para a equipe médica (diagnóstico de alto risco, exames pendentes)	<code>alerts</code>

Fonte: docs/architecture.md.

Todos os nós registram um evento em `logs/audit_log.jsonl` via `AuditLogger.log_flow_event`, garantindo rastreabilidade completa do fluxo. Um executor sequencial equivalente está disponível via `--no-langgraph`, para ambientes sem a dependência opcional instalada — sem alterar a interface pública usada pelo restante do sistema.

4.6 Segurança, Guardrails e Auditoria

O assistente nunca prescreve diretamente: `MedicalGuardrails.review_output` detecta e reescreve linguagem de ordem médica direta (ex.: "administre", "prescrevo", "inje") por uma formulação de sugestão, e todo texto final recebe o aviso obrigatório "[AVISO] ... requer validação de um médico responsável". Perguntas fora do escopo

seguro são bloqueadas por `check_input` antes de qualquer geração de resposta. Todo evento — pergunta, paciente, fontes, flags de guardrail — é persistido em `logs/audit_log.jsonl`, com `event_id` único, timestamp UTC e rastreabilidade completa.

O projeto foi desenhado para ser avaliável em qualquer máquina, sem exigir GPU, chave de API ou dependências pesadas: por padrão, usa TF-IDF local, um executor sequencial equivalente ao grafo LangGraph e o provider determinístico. Ao instalar as dependências opcionais (`requirements-langchain.txt` e `requirements-finetuning.txt`), o mesmo código passa a usar StateGraph real, índice FAISS e o modelo fine-tuned com adaptador LoRA — sem alterar a interface pública.

4.7 Interface Web e API

Além da CLI, o projeto disponibiliza uma interface de chat no navegador, com backend próprio (`src/webapp/server.py`, implementado apenas com a biblioteca padrão do Python, sem frameworks web) e front-end estático (`src/webapp/static/index.html`). No Windows, o script `iniciar_assistente.bat` verifica o ambiente virtual, sobe o backend na porta 8765 e abre o navegador automaticamente.

Tabela 6 — Endpoints do Backend

Endpoint	Método	Descrição
/	GET	Interface de chat
/api/patients	GET	Lista de pacientes de exemplo (para o seletor)
/api/ask	POST	{question, patient_id?} → resposta do assistente
/api/flow	POST	{patient_id} → executa o fluxo clínico automatizado
/api/logs?n=10	GET	Últimos eventos de auditoria

Fonte: `src/webapp/server.py`.

A interface oferece chat com fontes citadas, seletor de paciente (PAC-0001 a PAC-0006) para contextualização por prontuário, perguntas rápidas pré-validadas (incluindo uma que demonstra o bloqueio dos guardrails ao vivo) e um botão para executar o fluxo automatizado completo, exibindo diagnóstico, exames pendentes, sugestão de conduta e alertas diretamente no chat.

5 AVALIAÇÃO E RESULTADOS

5.1 Comandos de Execução

```
python -m src.main anonymize
python -m src.main build-dataset
python -m src.main finetune --max-steps 30
python -m src.main evaluate
python -m src.main ask "Quando iniciar antibiotico em suspeita de sepsis?" --patient-id PAC-0004
python -m src.main flow --patient-id PAC-0006
python -m src.main logs --n 10
python -m pytest -q
```

5.2 Comandos da CLI

Tabela 7 — Comandos Disponíveis (src/main.py)

Comando	Descrição
anonymize	Anonimiza data/raw/prontuarios_brutos.csv
build-dataset	Monta o dataset de fine-tuning a partir de protocolos, FAQs e laudos
finetune [--max-steps N]	Executa o fine-tuning LoRA (requer requirements-finetuning.txt)
evaluate	Avalia o modelo em perguntas de validação (hold-out)
ask "<pergunta>" [--patient-id ID] [--provider ...]	Pergunta ao assistente médico
flow --patient-id ID [--no-langgraph]	Executa o fluxo clínico automatizado
logs [--n N]	Mostra os últimos eventos de auditoria

Fonte: elaborado pelos autores.

5.3 Validação por Testes Automatizados

A suite de testes (pytest) cobre anonimização de prontuários, curadoria e montagem do dataset, recuperação RAG, guardrails de segurança, repositório de pacientes (SQLite), o pipeline do assistente médico e o fluxo clínico automatizado, totalizando **26 testes passando**, executáveis em segundos, sem GPU e sem chave de API.

5.4 Análise: Estratégia de Fallback como Garantia de Reprodutibilidade

A decisão arquitetural de manter, em cada camada, um modo padrão leve e determinístico (TF-IDF em vez de FAISS, executor sequencial em vez de StateGraph, provider determinístico em vez de modelo fine-tuned ou API paga) revelou-se central para a robustez do projeto: qualquer avaliador consegue rodar o sistema completo — incluindo os 26 testes — sem instalar dependências pesadas, sem GPU e sem custos de API, enquanto o mesmo código de produção pode ser escalado para os modos avançados sem alteração de interface.

5.5 Análise: Limite do Fine-tuning em CPU e Caminho para GPU

Uma primeira tentativa de treinar o modelo TinyLlama-1.1B nesta máquina, apenas em CPU (sem GPU), ultrapassou 6 horas de processamento sem completar sequer o primeiro dos dez passos de treino configurados, terminando com o desligamento espontâneo do computador por sobrecarga. Esse resultado negativo é reportado de forma transparente: a equipe optou por manter o distilgpt2 como modelo padrão do projeto, e o episódio evidenciou, na prática, a diferença de ordens de grandeza no custo computacional entre um modelo de 82M parâmetros (distilgpt2, ~90 segundos em CPU) e um modelo de 1,1B parâmetros (TinyLlama) sem aceleração de hardware. Em resposta, o pipeline de treino (`src/finetuning/train.py`) e o de inferência (`src/llm/provider.py`) foram tornados GPU-aware e agnósticos de fabricante: ambos detectam a GPU via `torch.cuda.is_available()` — verdadeiro tanto em CUDA/NVIDIA quanto em ROCm/AMD, já que o PyTorch ROCm expõe a mesma API — e ativam automaticamente `bf16/fp16` e `gradient_checkpointing` quando uma GPU está disponível, caindo para CPU caso contrário. Scripts dedicados (`scripts/install_torch_nvidia.sh` e `scripts/install_torch_amd.sh`) automatizam a instalação da build correta do PyTorch para cada fabricante, permitindo treinar o TinyLlama-1.1B em ~6 GB de VRAM sem quantização adicional.

6 RESULTADOS CONSOLIDADOS

O sistema entregue cobre integralmente os requisitos do enunciado da Fase 3:

- Pipeline de fine-tuning supervisionado com LoRA/PEFT, reproduzível em CPU;
- Suporte documentado a arquitetura LLaMA real (TinyLlama-1.1B) e ao Llama 3 oficial da Meta, para ambientes com GPU (NVIDIA/CUDA ou AMD/ROCm);
- Dataset de instruction-tuning com 71 exemplos curados, combinando dados sintéticos internos e uma amostra real do PubMedQA traduzida para PT-BR;
- Anonimização de prontuários com PII em duas camadas (estrutural + regex);
- Camada RAG testável sobre protocolos clínicos internos, com fallback leve e modo avançado (FAISS/LangChain);
- Assistente médico virtual (LangChain) que contextualiza respostas com o prontuário do paciente;
- Fluxo de decisão clínica automatizado (LangGraph) com seis nós auditáveis;
- Guardrails de segurança que nunca permitem prescrição direta, com aviso obrigatório de validação humana;
- Logging de auditoria estruturado (JSONL) com rastreabilidade completa de cada interação e cada nó do fluxo;
- Explicabilidade — toda resposta cita a fonte exata utilizada;
- Três providers de LLM intercambiáveis (determinístico, fine-tuned local, OpenAI), com seleção automática do modo mais seguro;
- CLI unificada, API HTTP e interface web de chat completa;
- 26 testes automatizados passando.

6.1 Métricas de Qualidade Consolidadas

Tabela 8 — Métricas Consolidadas do Projeto

Métrica	Resultado
Testes automatizados	26 passed (100% de aprovação)
Exemplos no dataset de fine-tuning	71 (17 protocolos + 10 FAQs + 4 laudos + 40 PubMedQA)
Overlap lexical (hold-out, baseline determinístico)	1,0 (3/3)
Tempo de fine-tuning (distilgpt2, CPU)	≈ 90 segundos
Tempo de tentativa de fine-tuning (TinyLlama-1.1B, CPU)	> 6h, computador desligou por sobrecarga, sem completar 1/10 passos
Providers de LLM implementados	3 (determinístico, fine-tuned local, OpenAI)
Nós do fluxo clínico (LangGraph)	6 (intake → exames → protocolo → sugestão → revisão → alertas)
Pacientes de exemplo	6 (PAC-0001 a PAC-0006)

Fonte: elaborado pelos autores, a partir dos artefatos gerados pelo projeto.

6.2 Conclusão

Os resultados demonstram a viabilidade de um assistente médico virtual que combina fine-tuning leve, RAG e orquestração de fluxos com LangGraph, mantendo, em todas as camadas, controles explícitos de segurança e auditoria. A estratégia de fallback determinístico garantiu reprodutibilidade total do sistema em qualquer ambiente, enquanto o suporte a treinamento em GPU (NVIDIA/CUDA e AMD/ROCm), já incorporado ao pipeline, deixa um caminho claro de evolução para um cenário de produção com modelos LLaMA reais e maior capacidade de geração.

REFERÊNCIAS

HU, E. J. et al. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685*, 2021.

LEWIS, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, v. 33, p. 9459–9474, 2020.

JIN, Q. et al. PubMedQA: A Dataset for Biomedical Research Question Answering. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019. Disponível em: <<https://pubmedqa.github.io/>>. Acesso em: set. 2026.

OPENAI. GPT-4o Technical Report. 2024. Disponível em: <<https://openai.com>>. Acesso em: set. 2026.

HUGGING FACE. Transformers Documentation e PEFT Library. 2024. Disponível em: <<https://huggingface.co/docs>>. Acesso em: set. 2026.

LANGCHAIN AI. LangChain Documentation. 2024. Disponível em: <<https://python.langchain.com>>. Acesso em: set. 2026.

LANGCHAIN AI. LangGraph Documentation. 2024. Disponível em: <<https://langchain-ai.github.io/langgraph/>>. Acesso em: set. 2026.

TINYLLAMA. TinyLlama-1.1B-Chat-v1.0 Model Card. Hugging Face, 2024. Disponível em: <<https://huggingface.co/TinyLlama>>. Acesso em: set. 2026.